

TÀI LIỆU HƯỚNG DẪN TRIỂN KHAI

GIAI ĐOẠN 2 — XÂY DỰNG APP FLUTTER DESKTOP & FILE HANDLING

Dự án: Second Brain App — Knowledge Base + Graph View + AI RAG Chatbot

Tài liệu bàn giao nội bộ nhóm — Phụ trách chính: Thành viên 2 (tham chiếu chung cho cả nhóm)

1. Mục Tiêu Của Giai Đoạn

Giai đoạn 2 xây dựng nền móng ứng dụng Desktop — nơi toàn bộ dữ liệu .md (do Thành viên 1 tạo ra) được đọc, hiển thị và quản lý. Đây là lớp trung gian mà cả Graph View (Giai đoạn 4) và AI RAG Chatbot (Giai đoạn 3) sẽ dựa vào, nên cần làm chắc và thống nhất cấu trúc dữ liệu ngay từ đầu.

- Khởi tạo và cấu hình project Flutter chạy được trên Windows/macOS.
- Đọc/quét toàn bộ file .md trong một thư mục cục bộ bằng dart:io.
- Hiển thị giao diện chia đôi màn hình (Split View): cây thư mục bên trái, nội dung Markdown render đẹp bên phải.
- Thiết kế Model dữ liệu & Service dùng chung để Thành viên 3, 4 tái sử dụng mà không phải viết lại logic đọc file.

2. Phạm Vi Công Việc & Đầu Ra (Deliverables)

- Project Flutter Desktop khởi tạo hoàn chỉnh, build chạy được trên ít nhất 1 nền tảng (Windows hoặc macOS).
- Module FileService: quét thư mục, đọc nội dung, trích liên kết [[]] cơ bản.
- Model dữ liệu chuẩn hóa NoteFile — “hộp đồng dữ liệu” chung cho cả nhóm.
- Giao diện Split View hoàn chỉnh: File Tree + Markdown Viewer, có thể chọn file để xem nội dung.
- Chức năng chọn & ghi nhớ thư mục Knowledge Base giữa các lần mở app.
- Tài liệu Service Contract — mô tả API/hàm mà Thành viên 3 và 4 có thể gọi trực tiếp.

3. Kiến Trúc & Luồng Dữ Liệu Của Module

Áp dụng mô hình phân lớp đơn giản (tương tự MVVM nhẹ) để dễ mở rộng và để các thành viên khác tích hợp:

```
[ UI / Widgets ]      FileTreeView, MarkdownViewer, SplitView
                      |
                      | lắng nghe (listen)
[ ViewModel / Provider ] NoteProvider (ChangeNotifier)
                      |
                      | gọi hàm
[ Service Layer ]      FileService (đọc, quét, parse link)
                      |
                      | đọc/ghi
[ Model ]              NoteFile (path, title, content, links...)
                      |
[ Local Folder ]      Thư mục chứa các file .md (do TV1 tạo)
```

Luồng hoạt động: người dùng chọn thư mục Knowledge Base → FileService quét toàn bộ file .md → chuyển thành danh sách `List<NoteFile>` → NoteProvider lưu state và thông báo cho UI → FileTreeView hiển thị danh sách, MarkdownViewer hiển thị nội dung file đang được chọn.

4. Chuẩn Bị Môi Trường

4.1. Cài đặt & bật hỗ trợ Desktop

```
# Kiểm tra Flutter đã cài đặt đúng phiên bản stable mới nhất
flutter doctor
```

```
# Bật hỗ trợ Desktop (chạy 1 lần trên máy dev)
```

```
flutter config --enable-windows-desktop
```

```
flutter config --enable-macos-desktop
```

```
# Khởi tạo project (nếu làm mới hoàn toàn)
```

```
flutter create --platforms=windows,macos second_brain_app
```

```
cd second_brain_app
```

```
flutter run -d windows # hoặc -d macos
```

4.2. Danh sách package cần cài đặt

Package	Mục đích sử dụng
flutter_markdown	Render nội dung Markdown (.md) ra giao diện đẹp, có style tùy chỉnh.
file_picker	Mở hộp thoại hệ điều hành để người dùng chọn thư mục Knowledge Base.
provider	Quản lý state toàn app (danh sách note, file đang chọn, trạng thái loading).
path	Nối/xử lý đường dẫn file an toàn, tương thích cả Windows và macOS.
shared_preferences	Lưu lại đường dẫn thư mục đã chọn để không phải chọn lại mỗi lần mở app.
watcher (tùy chọn)	Tự động phát hiện khi TV1 thêm/sửa file .md mới trong thư mục, không cần bấm Reload thủ công.

```
# pubspec.yaml
dependencies:
  flutter:
    sdk: flutter
  flutter_markdown: ^0.7.4
  file_picker: ^8.1.2
  provider: ^6.1.2
  path: ^1.9.0
  shared_preferences: ^2.3.2
  watcher: ^1.1.1 # tùy chọn
```

5. Cấu Trúc Thư Mục Project Đề Xuất

```
lib/
├─ main.dart
├─ models/
│   └─ note_file.dart # Model dữ liệu dùng chung
├─ services/
│   └─ file_service.dart # Quét thư mục, đọc file, parse link
├─ providers/
│   └─ note_provider.dart # State management (ChangeNotifier)
├─ screens/
│   └─ home_screen.dart # Màn hình chính (Split View)
├─ widgets/
│   └─ file_tree_view.dart # Danh sách file bên trái
│   └─ markdown_viewer.dart # Hiển thị nội dung bên phải
│   └─ split_view.dart # Bộ cục chia đôi màn hình
```

```
└─ utils/  
  └─ constants.dart          # Hằng số, key lưu shared_preferences
```

6. Model Dữ Liệu Dùng Chung — “Hợp Đồng” Giữa Các Thành Viên

Đây là phần quan trọng nhất để tránh xung đột code: Thành viên 3 (Graph View) và Thành viên 4 (AI RAG) sẽ dùng lại đúng model này thay vì tự viết logic đọc file riêng. Cần thống nhất sớm trong buổi họp nhóm đầu tiên.

```
// lib/models/note_file.dart
class NoteFile {
  final String path;          // Đường dẫn đầy đủ tới file .md
  final String fileName;      // Ví dụ: PRM393.md
  final String title;         // Tên hiển thị (mặc định = fileName không đuôi)
  final String rawContent;    // Toàn bộ nội dung file (markdown thô)
  final List<String> links;    // Các liên kết trích từ [...], ví dụ: ["PR0192"]
  final DateTime lastModified;

  NoteFile({
    required this.path,
    required this.fileName,
    required this.title,
    required this.rawContent,
    required this.links,
    required this.lastModified,
  });
}
```

7. Hướng Dẫn Triển Khai Từng Bước

Bước 1 — Khởi tạo & cấu hình Desktop

Thực hiện theo mục 4.1. Kiểm tra chạy được project rỗng trên desktop trước khi code tiếp.

Bước 2 — Xây dựng FileService (lỗi xử lý file)

Dùng `dart:io` để quét thư mục đệ quy, chỉ lấy file đuôi `.md`, đọc nội dung bằng UTF-8 (đảm bảo hiển thị đúng dấu tiếng Việt):

```
// lib/services/file_service.dart
import 'dart:convert';
import 'dart:io';
import '../models/note_file.dart';

class FileService {
  /// Quét toàn bộ file .md trong thư mục [folderPath] (bao gồm thư mục con)
  Future<List<NoteFile>> scanDirectory(String folderPath) async {
    final dir = Directory(folderPath);
    if (!await dir.exists()) return [];

    final List<NoteFile> notes = [];
    final entities = dir.listSync(recursive: true, followLinks: false);

    for (final entity in entities) {
      if (entity is File && entity.path.toLowerCase().endsWith('.md')) {
        try {
          final bytes = await entity.readAsBytes();
          final content = utf8.decode(bytes, allowMalformed: true);
          final fileName = entity.uri.pathSegments.last;
          final title = fileName.replaceAll('.md', '');
          final stat = await entity.stat();

          notes.add(NoteFile(
            path: entity.path,
            fileName: fileName,
            title: title,
            rawContent: content,
            links: [],
            lastModified: stat.lastModified,
          ));
        } catch (e) {
          // Xử lý lỗi khi đọc file
        }
      }
    }

    return notes;
  }
}
```

```

        title: title,
        rawContent: content,
        links: extractLinks(content),
        lastModified: stat.modified,
    ));
} catch (e) {
    // Bỏ qua file lỗi, log lại để debug thay vì crash cả app
    print('Lỗi đọc file ${entity.path}: $e');
}
}
}
return notes;
}

/// Trích các liên kết dạng [[TenFile]] trong nội dung markdown
List<String> extractLinks(String content) {
    final regex = RegExp(r'\[\[(.*?)\]\]');
    return regex.allMatches(content).map((m) => m.group(1)!.trim()).toList();
}
}

```

Bước 3 — Chọn & ghi nhớ thư mục Knowledge Base

Dùng `file_picker` để mở hộp thoại chọn thư mục, sau đó lưu lại đường dẫn bằng `shared_preferences` để lần sau mở app không phải chọn lại:

```

// Trích trong NoteProvider hoặc HomeScreen
import 'package:file_picker/file_picker.dart';
import 'package:shared_preferences/shared_preferences.dart';

Future<String?> pickKnowledgeBaseFolder() async {
    final path = await FilePicker.platform.getDirectoryPath();
    if (path != null) {
        final prefs = await SharedPreferences.getInstance();
        await prefs.setString('kb_folder_path', path);
    }
    return path;
}

```

Bước 4 — NoteProvider (quản lý state)

```

// lib/providers/note_provider.dart
import 'package:flutter/foundation.dart';
import '../models/note_file.dart';
import '../services/file_service.dart';

class NoteProvider extends ChangeNotifier {
    final FileService _fileService = FileService();

    List<NoteFile> notes = [];
    NoteFile? selectedNote;
    bool isLoading = false;

    Future<void> loadFromFolder(String folderPath) async {
        isLoading = true;
        notifyListeners();

        notes = await _fileService.scanDirectory(folderPath);
        selectedNote = notes.isNotEmpty ? notes.first : null;

        isLoading = false;
        notifyListeners();
    }

    void selectNote(NoteFile note) {

```

```
        selectedNote = note;
        notifyListeners();
    }

    NoteFile? getNoteByFileName(String fileName) {
        try {
            return notes.firstWhere((n) => n.fileName == fileName);
        } catch (_) {
            return null;
        }
    }
}
```

Bước 5 — Giao diện Split View

Bố cục chia đôi màn hình bằng Row + Expanded (tỉ lệ gợi ý 30/70). Có thể nâng cấp sau bằng package cho phép kéo-thả thanh chia:

```
// lib/widgets/split_view.dart
Row(
  children: [
    Expanded(flex: 3, child: FileTreeView()),
    VerticalDivider(width: 1),
    Expanded(flex: 7, child: MarkdownViewer()),
  ],
)
```

Bước 6 — File Tree View (danh sách bên trái)

```
// lib/widgets/file_tree_view.dart
Consumer<NoteProvider>({
  builder: (context, provider, _) {
    if (provider.isLoading) return Center(child: CircularProgressIndicator());
    return ListView.builder(
      itemCount: provider.notes.length,
      itemBuilder: (context, index) {
        final note = provider.notes[index];
        final isSelected = note.path == provider.selectedNote?.path;
        return ListTile(
          title: Text(note.title),
          selected: isSelected,
          onTap: () => provider.selectNote(note),
        );
      },
    );
  },
)
```

Bước 7 — Markdown Viewer (nội dung bên phải)

```
// lib/widgets/markdown_viewer.dart
Consumer<NoteProvider>({
  builder: (context, provider, _) {
    final note = provider.selectedNote;
    if (note == null) return Center(child: Text('Chưa chọn file nào'));
    return Markdown(
      data: note.rawContent,
      styleSheet: MarkdownStyleSheet(
        h1: TextStyle(fontSize: 22, fontWeight: FontWeight.bold),
        p: TextStyle(fontSize: 14, height: 1.5),
      ),
    );
  },
)
```

Bước 8 — Xử lý các trường hợp biên

- Thư mục rỗng hoặc không có file .md → hiển thị thông báo hướng dẫn thay vì màn hình trắng.
- File bị lỗi mã hóa (không phải UTF-8 chuẩn) → dùng allowMalformed: true khi decode để tránh crash app.
- Số lượng file lớn → dùng ListView.builder (đã áp dụng ở trên) để không load toàn bộ Widget cùng lúc.
- Đường dẫn khác nhau giữa Windows (\\) và macOS (/) → luôn dùng package path thay vì nối chuỗi thủ công.

Bước 9 — Kiểm thử & đóng gói thử

```
flutter run -d windows      # hoặc -d macos, kiểm tra chức năng đầy đủ  
flutter build windows       # hoặc macos, build thử bản release
```

- Test với bộ dữ liệu mẫu từ Thành viên 1 (ít nhất 10–15 file .md có liên kết chéo) trước khi bàn giao.
- Test riêng với file có tiếng Việt có dấu để chắc chắn không bị lỗi font khi hiển thị.

8. Service Contract — Bàn Giao Cho Thành Viên 3 & 4

Đây là danh sách hàm/API công khai mà FileService và NoteProvider cung cấp. Thành viên 3 (Graph View) và Thành viên 4 (AI RAG) nên gọi trực tiếp các hàm này thay vì viết lại logic đọc file, để tránh sai lệch dữ liệu giữa các module.

Hàm / Thuộc tính	Trả về	Dùng cho
FileService.scanDirectory(folderPath)	Future<List<NoteFile>>	Lấy toàn bộ note để build Graph hoặc nạp cho RAG.
FileService.extractLinks(content)	List<String>	TV3 dùng để vẽ Edge giữa các Node.
NoteProvider.notes	List<NoteFile>	Danh sách note hiện có trong state của app.
NoteProvider.getNoteByFileName(name)	NoteFile?	TV3 tra cứu note khi click vào 1 Node trên Graph.
NoteFile.links	List<String>	TV3 dùng trực tiếp, không cần parse lại nội dung.
NoteFile.rawContent	String	TV4 dùng làm dữ liệu đầu vào để embedding cho RAG.

Khuyến nghị: nên tách riêng model & service này thành một package/module dùng chung trong repo (ví dụ thư mục core/), để cả 4 thành viên cùng import mà không copy code qua lại.

9. Checklist Nghiệm Thu (Definition of Done)

- App khởi tạo và build/chạy được trên ít nhất 1 nền tảng desktop (Windows hoặc macOS).
- Chọn được thư mục Knowledge Base và đường dẫn được ghi nhớ giữa các lần mở app.
- Hiển thị đầy đủ danh sách file .md trong thư mục (kể cả thư mục con) ở File Tree.
- Click vào 1 file hiển thị đúng nội dung Markdown đã render, đúng định dạng tiếng Việt có dấu.
- Model NoteFile export đầy đủ field (path, title, content, links, lastModified).
- Đã xử lý các trường hợp biên: thư mục rỗng, file lỗi, số lượng file lớn.
- Tài liệu Service Contract đã được chia sẻ và thống nhất với Thành viên 3 và Thành viên 4.

10. Rủi Ro Thường Gặp & Cách Xử Lý

Rủi ro	Cách xử lý
TreeView phức tạp, mất nhiều thời gian nghiên cứu package.	Dùng ListView đơn giản trước để đảm bảo tiến độ, nâng cấp thành cây thư mục thật nếu còn thời gian.
Đường dẫn file khác nhau giữa Windows và macOS.	Luôn dùng package path để nối/xử lý đường dẫn, không nối chuỗi thủ công bằng "/" hoặc "\\".
File .md tiếng Việt có dấu hiển thị lỗi font/ký tự lạ.	Đọc file với UTF-8 (allowMalformed: true), kiểm tra font hiển thị hỗ trợ tiếng Việt.
Phải chờ dữ liệu thật từ Thành viên 1 mới code được UI.	Tự tạo 5-10 file .md mẫu có liên kết [[]] chéo nhau để phát triển song song, không phụ thuộc tiến độ TV1.
Thành viên 3, 4 viết lại logic đọc file riêng, gây lệch dữ liệu.	Thống nhất và chia sẻ Service Contract (mục 8) ngay từ tuần đầu tiên của giai đoạn.

11. Gợi Ý Mốc Thời Gian (2 Tuần)

Thời gian	Công việc chính
Tuần 1	Bước 1-4: Khởi tạo project desktop, xây dựng Model NoteFile, FileService (quét/đọc/parse link), chọn & lưu thư mục Knowledge Base.
Tuần 2	Bước 5-9: NoteProvider, giao diện Split View, File Tree View, Markdown Viewer, xử lý lỗi, testing & đóng gói thử, chốt Service Contract với TV3/TV4.

12. Tài Liệu Tham Khảo

- flutter_markdown — pub.dev/packages/flutter_markdown
- file_picker — pub.dev/packages/file_picker
- provider — pub.dev/packages/provider
- path — pub.dev/packages/path
- shared_preferences — pub.dev/packages/shared_preferences
- watcher — pub.dev/packages/watcher
- Flutter Desktop — docs.flutter.dev/platform-integration/desktop

□ **Lưu ý cuối:** Ưu tiên hoàn thành đúng “hợp đồng dữ liệu” (Model + Service Contract) trước khi tra soát giao diện. Đây là phần quyết định cả nhóm có ráp nối được với nhau đúng tiến độ ở Giai đoạn 3 và 4 hay không.